

교육관리 시스템 DB 오브젝트 설계

4조 김상준, 김요한, 권익성, 최유빈, 이동희

1. 전체 요약

스키마 조회 성능 확보를 위한 인덱스와, 자주 쓰는 조회/계산/트랜잭션 로직을 뷰·함수·프로시저로 정리했습니다.

구분	이름	목적/설명
인덱스	idx_attendance_student 외 9개	FK/검색/필터 조건 컬럼 성능 최적화
뷰	vwStudentAttendanceRate	학생별 출석을 집계
뷰	vwStudentCourseStatus	학생별 수강 과정 현황 조회
뷰	vwCourseEnrollmentSummary	과정별 수강신청 인원 집계
함수	fnCalcTotalScore	출석/필기/실기 점수 → 총점 계산
함수	fnGetAttendanceRate	특정 학생의 출석을 단건 조회
프로시저	prcEnrollStudent	수강 신청 처리(중복 체크 포함)
프로시저	prcCheckAttendance	당일 출결 등록(중복 방지)

2. 인덱스 (Index)

2-1. FK 조인용 인덱스

```
CREATE INDEX idx_attendance_student ON tblAttendance(studentNumber);
CREATE INDEX idx_enrollment_student ON tblEnrollment(studentNumber);
CREATE INDEX idx_enrollment_course ON tblEnrollment(courseNumber);
CREATE INDEX idx_grade_student ON tblGrade(studentNumber);
CREATE INDEX idx_grade_offering ON tblGrade(offeringNumber);
CREATE INDEX idx_offering_course ON tblOffering(courseNumber);
CREATE INDEX idx_course_teacher ON tblCourse(teacherNumber);
```

2-2. 검색/필터 조건용 인덱스

```
CREATE INDEX idx_attendance_date ON tblAttendance(attendanceDate);
CREATE INDEX idx_student_status ON tblStudent(studentAccountStatus);
CREATE INDEX idx_course_status ON tblCourse(courseStatus);
```

2-3. 복합 인덱스

```
CREATE INDEX idx_attendance_student_date
ON tblAttendance(studentNumber, attendanceDate);
```

3. 뷰 (View)

3-1. vwStudentAttendanceRate — 학생별 출석률

```
CREATE OR REPLACE VIEW vwStudentAttendanceRate AS
SELECT
    studentNumber,
    COUNT(*) AS totalDays,
    SUM(CASE WHEN attendanceType = '출석' THEN 1 ELSE 0 END) AS presentDays,
    ROUND(SUM(CASE WHEN attendanceType = '출석' THEN 1 ELSE 0 END)
        / COUNT(*) * 100, 1) AS attendanceRate
FROM tblAttendance
GROUP BY studentNumber;
```

3-2. vwStudentCourseStatus — 학생별 수강 과정 현황

```
CREATE OR REPLACE VIEW vwStudentCourseStatus AS
SELECT
    s.studentNumber, s.studentName,
    cc.courseName, e.enrollmentStatus, e.enrollmentDate
FROM tblStudent s
JOIN tblEnrollment e ON s.studentNumber = e.studentNumber
JOIN tblCourse c ON e.courseNumber = c.courseNumber
JOIN tblCourseCode cc ON c.courseCode = cc.courseCode;
```

3-3. vwCourseEnrollmentSummary — 과정별 신청 인원 집계

```
CREATE OR REPLACE VIEW vwCourseEnrollmentSummary AS
SELECT
    c.courseNumber, cc.courseName, c.courseStatus,
    COUNT(e.studentNumber) AS enrolledCount
FROM tblCourse c
JOIN tblCourseCode cc ON c.courseCode = cc.courseCode
LEFT JOIN tblEnrollment e ON c.courseNumber = e.courseNumber
GROUP BY c.courseNumber, cc.courseName, c.courseStatus;
```

4. 함수 (Function)

4-1. fnCalcTotalScore — 성적 총점 계산

출석 20% + 필기 40% + 실기 40%

```
CREATE OR REPLACE FUNCTION fnCalcTotalScore(  
    p_attendance NUMBER, p_written NUMBER, p_practical NUMBER  
) RETURN NUMBER IS  
BEGIN  
    RETURN ROUND(p_attendance * 0.2 + p_written * 0.4  
        + p_practical * 0.4, 1);  
END;  
/
```

4-2. fnGetAttendanceRate — 학생 출석률 단건 조회

```
CREATE OR REPLACE FUNCTION fnGetAttendanceRate(p_studentNumber NUMBER)  
RETURN NUMBER IS  
    v_total NUMBER;  
    v_present NUMBER;  
BEGIN  
    SELECT COUNT(*),  
        SUM(CASE WHEN attendanceType = '출석' THEN 1 ELSE 0 END)  
    INTO v_total, v_present  
    FROM tblAttendance  
    WHERE studentNumber = p_studentNumber;  
  
    IF v_total = 0 THEN RETURN 0; END IF;  
    RETURN ROUND(v_present / v_total * 100, 1);  
END;  
/
```

5. 프로시저 (Procedure)

5-1. prcEnrollStudent — 수강 신청 (중복 체크)

```
CREATE OR REPLACE PROCEDURE prcEnrollStudent(  
    p_studentNumber NUMBER, p_courseNumber NUMBER  
) IS  
    v_count NUMBER;  
BEGIN  
    SELECT COUNT(*) INTO v_count  
    FROM tblEnrollment  
    WHERE studentNumber = p_studentNumber AND courseNumber = p_courseNumber;  
  
    IF v_count = 0 THEN  
        INSERT INTO tblEnrollment(enrollmentDate, enrollmentStatus,  
            studentNumber, courseNumber)  
        VALUES (SYSDATE, '수강중', p_studentNumber, p_courseNumber);  
    END IF;  
END;
```

```

        COMMIT;
    ELSE
        RAISE_APPLICATION_ERROR(-20001, '이미 등록된 학생입니다.');
```

```

    END IF;
```

```

END;
```

```

/
```

5-2. prcCheckAttendance — 출석 체크 (당일 중복 방지)

```

CREATE OR REPLACE PROCEDURE prcCheckAttendance(
    p_studentNumber NUMBER, p_type VARCHAR2
) IS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM tblAttendance
    WHERE studentNumber = p_studentNumber
        AND attendanceDate = TRUNC(SYSDATE);

    IF v_count = 0 THEN
        INSERT INTO tblAttendance(attendanceNumber, attendanceDate,
                                   attendanceType, studentNumber)
        VALUES ((SELECT NVL(MAX(attendanceNumber),0)+1 FROM tblAttendance),
                TRUNC(SYSDATE), p_type, p_studentNumber);
        COMMIT;
    ELSE
        RAISE_APPLICATION_ERROR(-20002, '오늘 이미 출결 처리된 학생입니다.');
```

```

    END IF;
```

```

END;
```

```

/
```